Evolutionary Algorithm for Unstructured Heterogeneous Multi-agent Collaboration

Chak Lam Shek **Kristy Sakano** **Tyler Kitts**

Abstract

In the domain of heterogeneous multi-agent collaboration, our project addresses the challenge of learning policies that align with both individual agent properties and synchronize with their dynamic behaviors. In heterogeneous multi-agent evolutionary reinforcement learning (MERL), Quality Diversity (QD) is applied to search for diversity among agent types and form teams. However, existing approaches assume limited agent types, restricting teams to either fixed compositions or discrete sets. The QD method becomes ineffective when agents are selected from a continuous domain. Our contribution lies in formulating an unstructured heterogeneous multi-agent collaborative problem and proposing a MERL algorithm enhanced by the Large Language Model (LLM) for team formation. Simulations demonstrate some processes in training. This work advances MERL adaptability in scenarios involving team composition a continuous domain, offering a valuable contribution to heterogeneous multi-agent systems.

1 Introduction

In recent years, Multi-Agent Reinforcement Learning (MARL) has attracted significant attention [13, 21], finding applications in various domains such as robot navigation, financial investment, and social welfare. Despite improvements in algorithmic performance, MARL remains challenged by the complexity of exploration, particularly as the number of agents increases. The interactions between agents contribute to an exponential expansion of the state space. To address this challenge, a common strategy involves the use of evolutionary algorithms [14, 19, 1, 6, 5], which generate additional pairs of policies to systematically explore a broader state-space and discover optimal solutions. This approach proves beneficial in mitigating the increasing complexities associated with MARL's exploration challenges.

One of the pioneering frameworks in MARL is the sample-efficient method proposed by Shauharda et al [8]. This model strategically utilizes evolutionary algorithms to maximize sparse team-based objectives, coupled with policy gradients aimed at optimizing dense agent objectives. Expanding upon this foundational work, subsequent research endeavors extend the approach to accommodate the complexities of asymmetric and heterogeneous agents [9, 4, 3]. In these formulations, the components of heterogeneity are selectively chosen from a finite set of species. Notably, a key advancement [2] involves the segmentation of the training process into two distinct stages: Island and Mainland. During the Island stage, each species undergoes training for coupled policies, subsequently projected into a low-dimensional behavior space. The Mainland stage performs QD that leverages the diversity of behaviors and their distribution, selecting policies and agent compositions to generate training samples. The new policies are projected into the same behavior space, fostering iterative improvement across generations in both team formation and policies. This process ensures continual refinement in team dynamics and policy optimization with each subsequent generation.

The limitations of the existing approach become evident when addressing the formation of teams in a continuous domain due to the QD method. The existing method imposes restrictions on the number of species, consequently allowing the QD method [12, 7, 10] to generate a finite number of behavior spaces. However, when agents are chosen from a continuous domain, the generation of behavior spaces for all species becomes unfeasible due to the infinite nature of the space. This limitation shows the need for a different approach that can handle agents in continuous domain.

Preprint. Under review.

However, we can adapt LLMs to address this challenge. Recent research has highlighted the remarkable capability of LLMs in problem-solving through knowledge reasoning [18, 20, 17]. Therefore, we leverage LLMs by aligning it with the problem's description to facilitate team formation based on species properties and agent policy behavior space, where the species behavior space correlates with the species properties. This approach eliminates the necessity of generating an infinite number of behavior diagrams, offering a more efficient and scalable solution.

2 Problem

We formulate fully cooperative multi-agent tasks within the framework of a decentralized fully observable Markov decision process (Dec-MDP), incorporating a crucial aspect—individual budget constraints for each agent. The process is defined as the tuple $< \mathcal{N}, S, A, P, R, \gamma >$. Here, $\mathcal{N} = 1, ..., N$ denotes the set of N agents. Agents have the flexibility to select properties in a continuous domain$\phi = (\phi_1, \phi_2, ..., \phi_K)$ under the constraint $\sum_i c(\phi_i) \leq \delta$, where $c(\phi_i)$ represents the cost function associated with property ϕ_i. The joint state space S and joint action space A capture the collaborative dynamics, with the transitional function $P(s_t, a_t, \{\phi^n\}_{n=1}^N)$ navigating the current state, actions, and selected properties to the next state. Under the decentralized setting, each agent can decide their action independently. The reward function $R(s_t, a_t, st + 1)$ and the discount factor γ further contribute to the model's dynamic representation of fully cooperative interactions among agents, accounting for property selection, budget constraints, and the temporal aspects of decision-making. This objective aims to find optimal joint policies (π) and properties (ϕ) that yield the maximum cumulative reward over the time horizon,

$$\max_{\pi,\psi} \sum_{t=0}^{T} \gamma^t R(s_t, a_t, s_{t+1}) \tag{1}$$

$$st. \sum_{k=1}^{K} c(\phi_k^n) \leq \delta, \forall n \tag{2}$$

Consider a scenario where teams of rovers are deployed to collect iron from the Martian surface. Each rover in the team has the ability to customize its equipment, including parameters like engine power (ϕ_1 : max speed), truck size (ϕ_2 : capacity), and battery size (ϕ_3 : travel distance). However, each rover operates under a budget constraint, limiting the total cost of the customized equipment.

In this context, the objective is to maximize the cumulative reward over a specified time horizon. The reward is derived from the efficiency and effectiveness of each rover's equipment customization, influencing factors such as collection speed, load capacity, and travel distance. In this way, the optimization process aims to find the most effective combination of equipment customization for each rover, respecting their budget constraints, to maximize the overall success of the iron-collecting mission on the Martian surface.

3 Method

The algorithm follows an evolutionary approach to enhance team performance over successive generations. Initially, a population of M teams, each comprising N agents, is initialized with unique weights denoted as θ_N^M. The algorithm employs self-play to iteratively learn policies (π_{θ^M}) for each team.

In each generation, the teams undergo a rollout process, where fitness scores and replays are obtained by executing the learned policies. The algorithm then estimates the properties (ϕ_{est}^M) for all agents using the Property-LLM based on the obtained replays. The teams are ranked based on their fitness scores, and only the top-performing teams (top e) are retained.

For the retained teams, the algorithm updates their policies and critics. Subsequently, a second rollout is performed with exploration disabled, generating replays. The behavior of each agent is estimated using Policy-LLM based on these replays. Finally, for teams that were not retained, the algorithm samples an agent from a retained team and pairs it with a remaining agent using Pair-LLM. This pairing involves matching the estimated properties and the estimated behavior of the agents.

Algorithm 1

Initial a population of M teams with N agents each and initialize their weights θ_N^M
Perform Self play to learn π_{θ^M} for all team ▷ Self play
for generation = $1 : \infty$ **do**
  for team = 1: M **do**
    fitscore, Replay = Rollout($\pi_{\theta^M}, R, \mathbf{Explore} = True$)
    ϕ_{est}^M = Property-LLM(Replay)
    ▷ estimate the properties ϕ_{est}^M for all agents using the replay
  end for
  Rank the team according to the value of fitscore
  Keep the top e team
  for team $\in$ e **do**
    Update the policy (and critic) of the team
    , Replay = Rollout($\pi_{\theta^M}, R, \mathbf{Explore} = False$)
    $\phi_{behavior}^M$ = Policy-LLM(Replay)
    ▷ estimate the behavior for all agent using replay
  end for
  for team j $\notin$ K **do**
    Sample an agent from team $\in K$
    Pair the remaining agent using Pair-LLM($\phi_{est}^M, \phi_{behavior}^M$) by matching their properties and behavior
  end for
end for

4 Experimental Setup

4.1 PPO

We trained PPO agents in teams of 2 to move 2 categories of ingredients to cooking stations and deliver completed dishes. Within a team, each agent independently learned a policy and PPO agents were trained on two scenarios: cramped_corridor and bonus_order_test. Both scenarios required the agents to complete multiple orders with different combinations of onions and tomatoes, and rewards were only given when completed dishes were delivered. Once the agents were trained they were run again so their trajectories could be recorded. These trajectory files described the state at every time step, the actions taken at each time step, and the total rewards for the episode. The possible actions include: *move* in a direction and *interact* which is context sensitive to the tile directly in front of the agent.

After constructing the agent trajectory files, a pre-processing script extracted the action history of each Agent and categorized the agent's behavior at every timestep into six options: *move*, *carry onion*, *carry tomato*, *carry soup*, *fumble*, and *loiter*. The below table illustrates how the agent's behavior was classified.

Action	Classification Criteria
move	The action history has the 'move' state
carry onion	The agent has an onion as a held object*
carry tomato	The agent has a tomato as a held object*
carry soup	The agent has a tomato as a held object*
loiter	The agent does not have the 'move' or 'interact' state and is not holding any objects
fumble	The action history has the 'interact' state, but is not facing any interactable objects

Table 1: * If an agent is carrying an object, the action history may either be *move* or *interact*

4.2 LLM

To determine optimal joint policies and properties of the agents, we used a LLM that evaluated episodes and their associated rewards [15]. The LLM used in the first phase of our experimentation was Meta AI's Large Language Model Meta AI (Llama 2), a free and open source LLM trained with over 1 trillion tokens [16]. The specific flavor of Llama we used was llama-recipes, a companion repository designed for fine-tuning for domain adaptation.

The prompt provided to Llama 2 included the rules for our scenario, our desired response, and an example solution. Leveraging In-context Learning (ICL)'s Few-shot-Chain of Thought (CoT), we anticipated that providing a example solutions in the context of the scenario would enable the LLM to reason through intermediate steps to provide a correct response to the prompt [18]. A sample prompt is displayed in Figure 1.

Prompt

Example Scenario 1: Agent 1's action history is carry onion, carry tomato, carry soup. Agent 2's action history is to move, move, move. The reward is 100.
Example Answer 1: Agent 1's policy is to carry onion, carry tomato, and carry soup. Agent 2's policy is to do nothing.

Example Scenario 2: Agent 1's action history is carry onion, carry onion, carry onion, carry soup. Agent 2's action history is carry tomato, carry tomato, carry tomato, carry soup. The reward is 100.
Example Answer 2: Agent 1'spolicy is to carry onion. Agent 2's policy is to carry tomato. Since the carry soup policy is equally distributed between Agent 1 and Agent 2, we may assign the policy of 'carry soup' to either agent.

Context: Two agents are playing a game where they must carry onion, tomato, and soup. The game is played for several episodes and in each episode, a reward is given.

Rule 1: All policies must be assigned. The available policies are 'carry onion', 'carry tomato', and 'carry soup'.
Rule 2: A policy cannot be assigned to more than 1 agent. If there is an equal assignment scenario, randomly select one agent and give them the policy.
Rule 2: An agent may have zero, one, more than one, or all food policies assigned to them.
Rule 3: An episode is a series of actions each agent took in this cooperative multi-agent game. The success or failure of the episode is summarized in the reward. The higher the reward, the more weight it should have in policy assignment.
Rule 4: To assign a policy to an agent, use the frequency of the action in an episode and the reward of the episode.

Use the below scenarios to generate your response.
Episode 0: Reward is 100
Agent 1 action history: carry onion, carry onion, carry onion, carry soup
Agent 2 action history: carry tomato, carry tomato, carry tomato, carry soup

Episode 1: Reward is 50
Agent 1 action history: carry tomato, carry tomato, carry tomato, carry soup
Agent 2 action history: carry onion, carry onion, carry onion, carry soup

Episode 2: Reward is 100
Agent 1 action history: carry soup, carry soup, carry soup
Agent 2 action history: carry soup, carry soup, carry soup

What are the optimal food policies for each agent to take in the next episode, while adhering to all the rules? Remember that all policies must be assigned to the agents, so one agent will have at least two policies. Give me ONE policy summarizing all the episodes with explanation.

Figure 1: LLM Prompt for the **Bad Agent 2 Scenario**

The second phase of the experimentation evaluated on OpenAI's Chat Generative Pre-trained Transformer (ChatGPT) [11]. The specific version was the Web API ChatGPT version GPT-3.5. The same prompt was provided for both Llama 2 and ChatGPT.

While pre-processed action history scripts were input into Llama 2, the Web API version of ChatGPT was not able to read files; therefore, the maximum prompt word limit was the limiting factor for input data. Instead, truncated and manually selected scenarios were used as the input for ChatGPT. A future expansion would be to use the API version of ChatGPT to read pre-processed action history scripts.

Two specific scenarios were tested. First, we tested whether a LLM could return the correct policy for a simple scenario where rewards were maximized if Agent 1's action history was only *carry onions* and Agent 2's action history was only *carry tomatoes*, and both agents had the same rewards for the *carry soup* action. The correct policy distribution would be to assign Agent 1 the *carry onions* policy and Agent 2 the *carry tomatoes* policy, and the *carry soup* policy could be randomly assigned to ether agent; this prompt is labeled as the **Equal Assignment Scenario** and illustrated in Figure 1.

The second scenario had maximized rewards in episodes where Agent 1's action history was *carry onions*, *carry tomatoes*, and *carry soup* and Agent 2's action history was comprised of *move*, *loiter*, and *fumble*. The correct policy distribution would be to assign Agent 1 all the food policies and assign Agent 2 no policies. This prompt is labeled as the **Bad Agent 2 Scenario** illustrated in the **Appendix**.

5 Results

5.1 PPO Results

The above figures show the mean reward at the end of an episode for the trained agent in the cramped corridor and bonus order test scenario.The agents were run an additional time in each scenario so their trajectory files could be used as input files to the Llama 2 LLM. As stated previously, the Web API version of ChatGPT does not permit input files and manually constructed action histories and their associated rewards were input to ChatGPT.

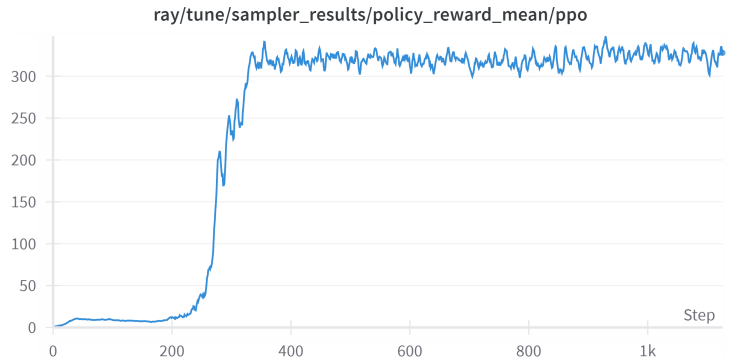

Figure 2: Mean reward at at the end of an episode in the cramped corridor environment vs number of training iterations

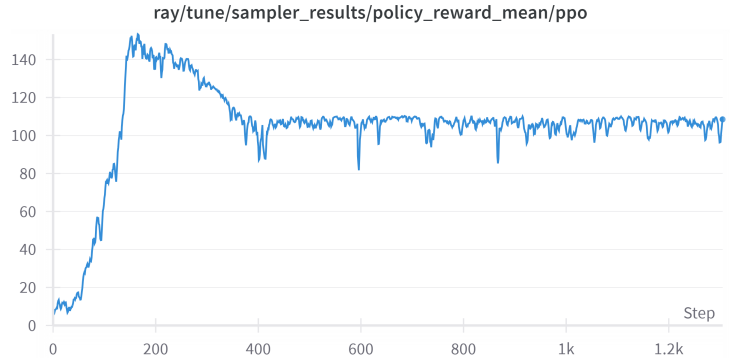

Figure 3: Mean reward at at the end of an episode in the bonus order test environment vs number of training iterations

5.2 LLM Results

Both Llama 2 and ChatGPT provided text-based responses to the prompt. Using the same prompt in both LLMs, querying the LLM resulted in different recommended policies. Sample responses are provided in Figure 4.

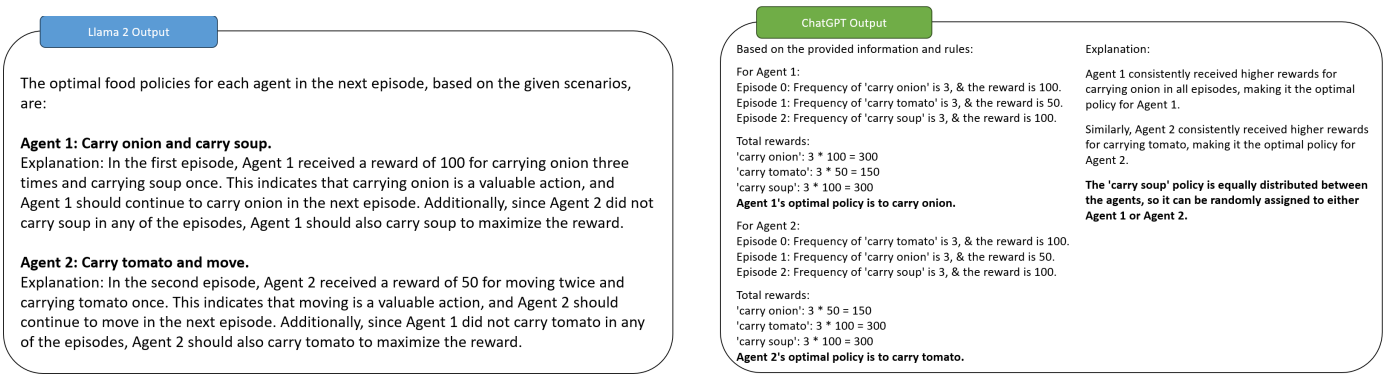

Figure 4: Sample Responses for Llama 2 and ChatGPT v3.5 in the **Equal Assignment Scenario**

Both the **Equal Assignment Scenario** and the **Bad Agent 2 Scenario** were queried 10 times each to both Llama 2 and ChatGPT. A summary of the results are detailed in Table 2.

	Scenario	Correct	Partially Correct	Wrong
Equal Assignment Scenario	Llama 2	0	10	0
	ChatGPT	1	7	2
Bad Agent 2 Scenario	Llama 2	0	10	0
	ChatGPT	4	4	2

Table 2: Results from 10 queries to ChatGPT and Llama 2

A partially correct assignment manifests in instances where either a) not all food policies are allocated to agents or b) a food policy is randomly allocated to both agents. Although the latter scenario of random assignment is undesirable, particularly in contexts where policy assignments ought to be clearly discrete, it serves as an indicative signal that the Language Model (LLM) exhibited a lack of confidence in the assignment, thereby opting to circumvent an incorrect allocation.

6 Discussion

As observed in Table 2, ChatGPT provided highly variable responses for clearly defined scenarios. Due to the observed variability in responses in instances of well-defined scenarios, we hypothesize that the introduction of less precisely defined scenarios may lead to an increased incidence of inaccurate responses. A comprehensive examination involving a broader spectrum of diverse scenarios is imperative to assess the frequency of which both LLMs generate incorrect responses. Furthermore, a more in-depth investigation is warranted in the domain of explainable AI to ascertain the level of confidence inherent in the responses provided by each LLM.

Furthermore, it was noted that the outputs generated by Llama 2 consistently yielded identical policy assignments and explanations. This phenomenon may be ascribed to a low temperature parameter within the offline API version of Llama 2. The temperature parameter - a tunable variable - diversifies the output response and consequently, the contextual adherence. A low temperature setting results in a focused and deterministic LLM response, while a higher temperature setting introduces variability. It is postulated that the temperature parameter for Llama 2 may have been set to 0, preventing any diversification in results, whereas ChatGPT likely operated with a considerably higher temperature setting. A potential future avenue involves comparative analysis across a spectrum of temperature gradients to optimize for both variation and accuracy in results.

During the process of prompt refinement, we encountered instances of inaccuracies in responses generated by both ChatGPT and Llama 2. Despite the explicit articulation of rules and evaluation of our scenarios with human evaluators (to successful policy classifications), our LLMs consistently produced responses that were confidently incorrect. The iterative development of the final prompt involved numerous revisions and refinements, culminating in the version documented in Figure 1. This iterative process illustrates the need for meticulous construction and beta testing of prompt engineering, particularly on trivial and well-defined scenarios prior to their application in large-scale testing.

7 Future Work

Owing to the time constraints of this project, we were unable to utilize the LLM outputs to incentivize or disincentivize different reward structures for each agent. As stated in the **Introduction**, the utility of the LLM was intended to sharpen the difference between Agent 1 and Agent 2's reward functions, which will subsequently affect the policies that each agent developed when training PPO agents in the next round. In theory, an altered reward function would effect a policy that would motivate different actions for each agents, and anticipatorily increase the joint reward and joint performance of both agents together.

A notable advantage inherent in LLMs lies in their capacity to accommodate substantial variability in the abstraction of the input trajectory files. For instance, a real-life action history file may have hundreds or thousands of actions, and likely may encompass multiple synonymous names. In a given scenario, actions such as *carry onion*, *pick up onion*, *hold white onions*, and *holding many onions* should all refer to the same action, but may be mis-classified as unique actions with a typical data processing algorithms. A LLM is adept at discerning the equivalence of these expressions and can theoretically handle action abstraction. A future avenue would explore synonymous actions and explore the extent to which a LLM would be able to correctly correlate or group actions together to provide a single comprehensive policy.

8 Contribution

- Chak Lam : Introduced the idea and the algorithm. Set up the Umiacs environment for overcook and Llama2. Debugged and modefied the LLM and PPO scripts for the simulation.
- Kristy : Wrote pre-processing script to obtain action classifications across episodes as LLM input files. Developed prompts for Llama 2 and ChatGPT using ICL and rule specifications & subsequent LL experimentation.
- Tyler : trained PPO agents in teams of 2 using handcrafted hyper parameters for 2 scenarios in the overcooked environment. He also modified Kristy's pre-processing script to include the total rewards gained each episode in the LLM input files.

9 Reference

References

[1] Ayhan Alp Aydeniz, Robert Loftin, and Kagan Tumer. Novelty seeking multiagent evolutionary reinforcement learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 402–410, 2023.

[2] Gaurav Dixit, Everardo Gonzalez, and Kagan Tumer. Diversifying behaviors for learning in asymmetric multiagent systems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 350–358, 2022.

[3] Gaurav Dixit and Kagan Tumer. Balancing teams with quality-diversity for heterogeneous multiagent coordination. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 236–239, 2022.

[4] Gaurav Dixit and Kagan Tumer. Learning inter-agent synergies in asymmetric multiagent systems. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pages 1569–1577, 2023.

[5] Gaurav Dixit and Kagan Tumer. Learning synergies for multi-objective optimization in asymmetric multiagent systems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 447–455, 2023.

[6] Yunfei Du, Yin Wang, Ya Cong, Weihao Jiang, and Shiliang Pu. Evolution strategies enhanced complex multiagent coordination. In *2023 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9, 2023.

[7] Tanmay Gangwani, Jian Peng, and Yuan Zhou. Harnessing distribution ratio estimators for learning agents with quality and diversity. In *Conference on Robot Learning*, pages 2206–2215. PMLR, 2021.

[8] Shauharda Khadka, Somdeb Majumdar, Santiago Miret, Stephen McAleer, and Kagan Tumer. Evolutionary reinforcement learning for sample-efficient multiagent coordination, 2020.

[9] Pengyi Li, Jianye Hao, Hongyao Tang, Yan Zheng, and Xian Fu. Race: improve multi-agent reinforcement learning with representation asymmetry and collaborative evolution. In *International Conference on Machine Learning*, pages 19490–19503. PMLR, 2023.

[10] Wenhao Li, Baoxiang Wang, Shanchao Yang, and Hongyuan Zha. Diverse policy optimization for structured action space. *arXiv preprint arXiv:2302.11917*, 2023.

[11] OpenAI. Chatgpt v3.5, 2023.

[12] Jack Parker-Holder, Aldo Pacchiano, Krzysztof M Choromanski, and Stephen J Roberts. Effective diversity in population based reinforcement learning. *Advances in Neural Information Processing Systems*, 33:18050–18062, 2020.

[13] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning, 2018.

[14] Enna Sachdeva, Shauharda Khadka, Somdeb Majumdar, and Kagan Tumer. Maedys: Multiagent evolution via dynamic skill selection. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 163–171, 2021.

[15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation models, 2023.

[16] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[17] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023.

[18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

[19] Xiaoqiang Wu, Qingling Zhu, Qiuzhen Lin, Jianqiang Li, Jianyong Chen, and Zhong Ming. An efficient evaluation mechanism for evolutionary reinforcement learning. In *International Conference on Intelligent Computing*, pages 41–50. Springer, 2022.

[20] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.

[21] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative, multi-agent games, 2022.

10 Appendix

Prompt

Example Scenario 1: Agent 1's action history is carry onion, carry tomato, carry soup. Agent 2's action history is to move, move, move. The reward is 100.
Example Answer 1: Agent 1's policy is to carry onion, carry tomato, and carry soup. Agent 2's policy is to do nothing.

Example Scenario 2: Agent 1's action history is carry onion, carry onion, carry onion, carry soup. Agent 2's action history is carry tomato, carry tomato, carry tomato, carry soup. The reward is 100.
Example Answer 2: Agent 1'spolicy is to carry onion. Agent 2's policy is to carry tomato. Since the carry soup policy is equally distributed between Agent 1 and Agent 2, we may assign the policy of 'carry soup' to either agent.

Context: Two agents are playing a game where they must carry onion, tomato, and soup. The game is played for several episodes and in each episode, a reward is given.

Rule 1: All policies must be assigned. The available policies are 'carry onion', 'carry tomato', and 'carry soup'.
Rule 2: A policy cannot be assigned to more than 1 agent. If there is an equal assignment scenario, randomly select one agent and give them the policy.
Rule 2: An agent may have zero, one, more than one, or all food policies assigned to them.
Rule 3: An episode is a series of actions each agent took in this cooperative multi-agent game. The success or failure of the episode is summarized in the reward. The higher the reward, the more weight it should have in policy assignment.
Rule 4: To assign a policy to an agent, use the frequency of the action in an episode and the reward of the episode.

Use the below scenarios to generate your response.
Episode 0: Reward is 100
Agent 1 action history: carry onion, carry onion, carry onion, carry soup
Agent 2 action history: move, move, move, move

Episode 1: Reward is 50
Agent 1 action history: carry tomato, carry tomato, carry tomato, carry soup
Agent 2 action history: loiter, move, loiter, loiter

Episode 2: Reward is 0
Agent 1 action history: fumble, loiter, move, move
Agent 2 action history: carry onion, carry tomato, carry soup

What are the optimal food policies for each agent to take in the next episode, while adhering to all the rules? Remember that all policies must be assigned to the agents, so one agent will have at least two policies. Give me ONE policy summarizing all the episodes with explanation.

Figure 5: LLM Prompt for the **Bad Agent 2 Scenario**